

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

ЛАБОРАТОРНАЯ РАБОТА № 1

по курсу «Компьютерная графика»

Основы 3D-графики

Вариант № 4 — усечённый правильный тетраэдр

Выполнил: Н. В. Храмов

Группа: М8О-308Б-24

Преподаватель: В. Д. Бахарев

Москва, 2026

1 УСЛОВИЕ

Цель лабораторной работы — познакомиться с построением простых трёхмерных объектов, их отображением на двумерной плоскости и интерактивным изменением проекции и ориентации фигуры. Работа выполнена на основе предоставленного проекта Vulkan Starter App на языке C++20.

Согласно варианту № 4 необходимо построить усечённый правильный тетраэдр, гранями которого являются правильные треугольники и правильные шестиугольники. Особое внимание уделяется правильности геометрии: грани должны быть плоскими, а длины всех рёбер — одинаковыми.

Программа должна использовать Vulkan, GLFW и Dear ImGui, работать в реальном времени и позволять пользователю динамически менять проекцию и трансформации объекта. Отображение фигуры должно учитывать выбранную проекцию и обеспечивать взаимодействие через интерфейс и устройства ввода.

В реализованном приложении предусмотрены перспективная и ортографическая проекции, поворот вокруг трёх координатных осей, изменение масштаба отображения, каркасный режим и автоматическое вращение. Переключение режимов выполняется без перезапуска программы.

В задании отдельно перечислены дополнительные возможности для повышения оценки. В данном отчёте рассматривается базовый вариант и реализованное переключение проекций. Перенос и независимое растяжение объекта по осям, сложная траектория движения, настройка цвета и процедурная окраска вершин, а также несколько объектов с отдельными наборами дескрипторов не реализованы. Автоматическое вращение не является выполнением задания о сложной траектории.

2 МЕТОД РЕШЕНИЯ

2.1 Построение геометрии

Для построения фигуры используется правильный тетраэдр с центром в начале координат. Его четыре вершины выбраны симметрично, поэтому расстояния между любыми двумя исходными вершинами одинаковы. На каждом ребре отмечаются две точки: на расстоянии одной трети длины ребра от каждого из его концов.

В программе исходные вершины имеют координаты $A_0 = (1, 1, 1)$, $A_1 = (1, -1, -1)$, $A_2 = (-1, 1, -1)$ и $A_3 = (-1, -1, 1)$. Новая вершина на направленном ребре от A_i к A_j вычисляется как

$$P_{ij} = \frac{2A_i + A_j}{3}, \quad i \neq j. \quad (1)$$

Здесь $i, j \in \{0, 1, 2, 3\}$. Для каждого ребра используются оба направления, поэтому получаются 12 различных точек. Длина исходного ребра равна $2\sqrt{2}$, а длина любого ребра усечённой фигуры — $2\sqrt{2}/3$.

Вершины исходного тетраэдра отсекаются плоскостями, проходящими через отмеченные точки. Каждая из четырёх отсечённых вершин образует новую треугольную грань. Каждая исходная треугольная грань после удаления трёх углов превращается в шестиугольник. Усечение именно на одной трети ребра обеспечивает одинаковую длину сторон треугольников и шестиугольников.

Полученная модель содержит 12 вершин, 18 рёбер и 8 граней: четыре правильных треугольника и четыре правильных шестиугольника. Фигура является выпуклой. Каждое ребро принадлежит двум граням, а в каждой вершине сходятся три ребра.

Геометрия хранится отдельно от интерфейса в файлах `tetrahedron.hpp` и `tetrahedron.cpp`. Модель представлена массивом пространственных вершин, списком граней и списком уникальных рёбер. Для каждой грани задаётся нормаль, направленная наружу. Вершины

грани упорядочиваются по кругу относительно её центра, что обеспечивает последовательный обход контура и корректную заливку.

2.2 Поворот и проецирование

Перед отображением все вершины последовательно поворачиваются вокруг осей X, Y и Z. Центр вращения совпадает с центром фигуры. Углы задаются ползунками интерфейса или изменяются перетаскиванием мыши. Поворот сохраняет форму, длины рёбер и расстояния между вершинами.

Для вектора-столбца координат порядок преобразований записывается как

$$p' = R_z(\gamma)R_y(\beta)R_x(\alpha)p, \quad (2)$$

где α, β, γ — углы поворота вокруг осей X, Y и Z. Матрица справа применяется первой. Например, поворот вокруг X имеет вид

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}. \quad (3)$$

В коде те же операции выполняются непосредственно над компонентами вектора, без хранения отдельных матриц.

Камера расположена на положительной полуоси Z и направлена к центру фигуры. Плоскость проекции проходит через начало координат. Положение камеры фиксировано, а пользователь изменяет ориентацию объекта. Нормали граней поворачиваются вместе с вершинами, чтобы проверка видимости и освещение соответствовали текущему положению модели.

Ортографическая проекция переносит горизонтальную и вертикальную составляющие положения вершины на плоскость экрана без уменьшения удалённых частей фигуры. Размер изображения не зависит от глубины точки. Такой режим удобен для изучения формы и сравнения пропорций.

Для точки (x, y, z) после поворота ортографическая проекция задаётся выражениями

$$x_{\text{pr}} = x, \quad y_{\text{pr}} = y. \quad (4)$$

При перспективной проекции учитывается расстояние от вершины до камеры. Ближние части объекта выглядят крупнее, а дальние — меньше. Это создаёт ощущение глубины. В программе проекция вычисляется непосредственно для каждой вершины на центральном процессоре; отдельные матрицы проекции в шейдер не передаются.

При расположении камеры в точке $(0, 0, d)$ и плоскости проекции $z = 0$ используются формулы

$$x_{\text{pr}} = \frac{dx}{d - z}, \quad y_{\text{pr}} = \frac{dy}{d - z}, \quad d = 6. \quad (5)$$

Все вершины при допустимых поворотах находятся перед камерой, поэтому $d - z > 0$. В экранной системе результат умножается на общий масштаб, смещается к центру области рисования, а вертикальная координата меняет знак.

2.3 Отображение и взаимодействие

После проецирования координаты переводятся в координаты области рисования: начало координат размещается в центре, вертикальное направление согласуется с экранной системой, а к изображению применяется общий масштаб. Масштаб определяется меньшей стороной области рисования, поэтому при изменении размеров окна пропорции фигуры сохраняются. Увеличенное изображение ограничивается границами этой области.

В режиме заливки отображаются только грани, обращённые к камере. Для определения видимости учитываются направление нормали грани и направление на камеру. Для ортографической проекции направление наблюдения одинаково для всех граней, а для перспективной определяется

относительно центра конкретной грани. Это позволяет корректно менять видимые поверхности при вращении.

Видимые грани сортируются от дальних к ближним и передаются для рисования как выпуклые двумерные полигоны. Для текущего одиночного выпуклого объекта отсечение обратных граней достаточно для устранения скрытых поверхностей. Буфер глубины для полигонов модели не используется. Применение этого подхода к нескольким пересекающимся объектам потребовало бы другого решения.

Треугольные грани окрашены в оранжевый цвет, а шестиугольные — в синий. Яркость зависит от ориентации грани относительно фиксированного источника света. Контуры выделены тёмными линиями. В каркасном режиме выводятся все 18 рёбер, включая скрытые, что позволяет рассмотреть внутреннюю структуру проекции.

Поворот по трём осям регулируется отдельными ползунками. Перетаскивание в области рисования изменяет поворот вокруг осей X и Y. Колёсико мыши и ползунок Zoom меняют масштаб отображения. Кнопка Reset view восстанавливает исходные углы и масштаб. Флажки переключают проекцию, каркас и автоматическое вращение. Скорость автоматического вращения учитывает время между кадрами.

2.4 Использование Vulkan, GLFW и Dear ImGui

GLFW создаёт окно, обрабатывает события клавиатуры и мыши и предоставляет поверхность для вывода Vulkan. Изменение размеров окна передаётся графической подсистеме. Dear ImGui формирует элементы управления и список команд рисования, а его Vulkan-модуль выводит интерфейс и спроецированную модель на экран.

Графическая подсистема использует экземпляр Vulkan, физическое и логическое устройства, поверхность окна, цепочку обмена изображениями, представления изображений и кадровые буферы. Для записи и выполнения команд используются пулы команд, буферы команд и графическая очередь.

Пул дескрипторов обслуживает ресурсы Dear ImGui. Семафоры и ограждение синхронизируют получение изображения, выполнение команд и показ кадра.

Каждый кадр начинается с обработки событий и обновления интерфейса. Затем вычисляются новые положения вершин и их проекции. Основной проход Vulkan очищает цветное изображение и буфер глубины; следующий проход рисует Dear ImGui и модель. После выполнения команд изображение передаётся на показ. Пользовательские 3D-шейдеры и отдельный графический конвейер для тетраэдра в этой реализации не создаются.

3 РЕЗУЛЬТАТЫ

В результате получено интерактивное приложение, отображающее усечённый правильный тетраэдр. В обычном режиме различимы треугольные и шестиугольные грани, их контуры и изменение яркости при повороте. Каркасный режим показывает полный набор рёбер и позволяет проследить связи между вершинами.

Проверка геометрии выполнена отдельной программой `tetrahedron_test.cpp`. Она проверяет количество вершин, рёбер и граней, наличие четырёх граней каждого типа, одинаковую длину рёбер, плоскостность и правильность граней, выпуклость их контуров и направление нормалей. Дополнительно проверены принадлежность каждого ребра двум граням, наличие трёх рёбер в каждой вершине и сохранение расстояния до центра после поворота. Все проверки завершились успешно.

Проект успешно собран в конфигурации Debug средствами CMake и Ninja. При запуске после исправления записи основного буфера команд сообщения об ошибках Vulkan не появились в наблюдавшемся сеансе. Предупреждения компилятора в сторонней библиотеке VulkanMemoryAllocator не препятствовали сборке.

Работа обеих проекций и каркасного режима проверена на запущенном приложении. При смене перспективной проекции на ортографическую меняется вид фигуры, но сохраняются её геометрия и текущая ориентация.

Ползунки и управление мышью позволяют изменять вид без перезапуска. Отображение пересчитывается в основном цикле программы; количественное измерение частоты кадров не проводилось.

Реализация использует все три обязательные библиотеки. Требование динамической смены проекции выполнено переключателем в интерфейсе. Взаимодействие с объектом обеспечено поворотом, масштабом отображения и режимами визуализации. Изменение масштаба отображения не является независимым растяжением самой модели по координатным осям.

Обнаруженное при первоначальном запуске падение было связано с отправкой незаписанного буфера команд и отсутствием подготовки изображения к следующему проходу рисования. В функцию `render` добавлены запись команд и основной проход очистки. Кроме того, исправлено условие, которое ошибочно выводило сообщение о неудачном показе кадра при успешном результате.

3.1 Примеры работы программы

На рисунке 1 показана перспективная проекция с заливкой видимых граней. Оранжевым выделены треугольники, синим — шестиугольники. На рисунке 2 включены ортографическая проекция и каркас: видны все рёбра, включая скрытые. Ориентация объекта и масштаб в обоих случаях одинаковы. Скриншоты получены из работающего приложения.

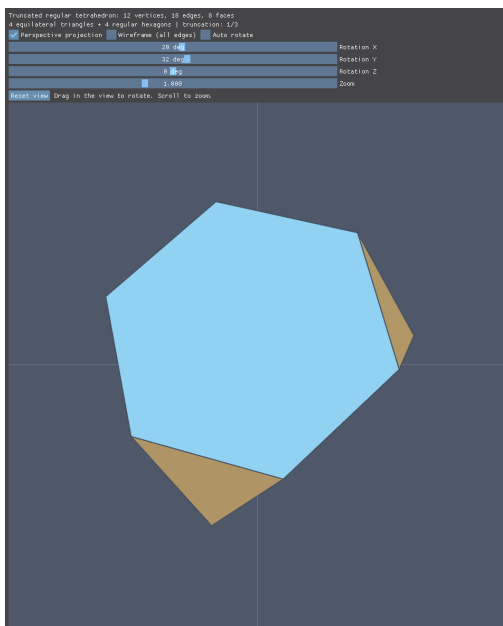


Рисунок 1 – Перспективная проекция с заливкой граней

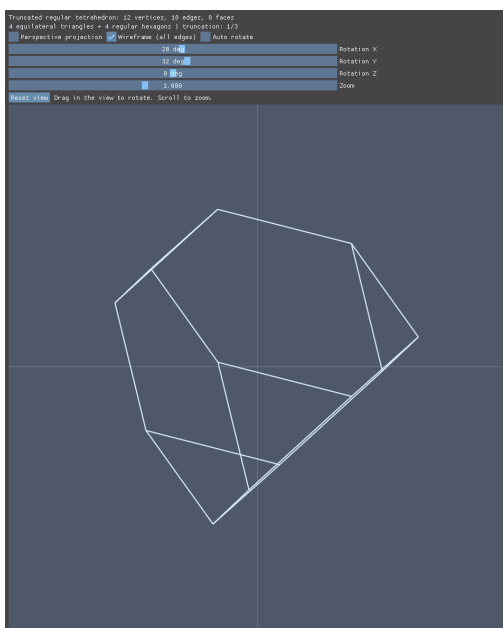


Рисунок 2 – Ортографическая проекция в каркасном режиме

4 ВЫВОДЫ

В ходе лабораторной работы построена корректная геометрическая модель усечённого правильного тетраэдра и реализовано её отображение на двумерной плоскости. Рассмотрены различия перспективной и ортографической проекций, поворот пространственных вершин, определение видимых граней и преобразование координат в экранную систему.

Получены практические навыки работы со стартовым проектом Vulkan, обработки окна и ввода через GLFW и создания интерактивного интерфейса с Dear ImGui. Проверены правильность граней, замкнутость поверхности и сохранение геометрии при повороте. Освоены запись буферов команд и последовательное выполнение проходов рисования.

Базовый вариант № 4 выполнен. Переключение проекций, вращение и масштаб отображения доступны в реальном времени. Выбранный способ визуализации достаточен для одного выпуклого объекта. Для дальнейшего расширения можно добавить трансформации положения и растяжения, управление цветом, сложную анимацию и вывод нескольких моделей с отдельными ресурсами.